PROSPECTOR - PROJECT PLAN PART 2

TEAM ROLES, TASK BREAKDOWN & TIMELINE

---

10. TEAM ROLE ASSIGNMENTS

MEMBER 1: Data & Astrodynamics Engineer

Owns: `backend/app/services/jpl_client.py`, `nhats_client.py`, `cad_client.py`, `delta_v_calc.py`, `mission_planner.py`, `data_sync.py`, `utils/orbital_math.py`, `utils/constants.py`, `utils/unit_converter.py`, `routers/asteroids.py`, `routers/trajectories.py`, `routers/mission.py`, `models/asteroid.py`, `models/orbital_elements.py`, `models/trajectory.py`

Week 1 Tasks

- [T1.1] Create `services/jpl_client.py` - wrapper class for SBDB API
  - Method: `get_asteroid(designation)` -> returns orbit + physical params
  - Method: `search_asteroids(query)` -> returns list of matches
  - Method: `query_asteroids(filters)` -> bulk query with sb-class, sb-kind filters
  - Handle HTTP errors, retries (3 attempts), timeout (15s)
  - Parse JSON response into Python dataclasses
- [T1.2] Create `services/nhats_client.py` - wrapper for NHATS API
  - Method: `get_accessible_asteroids(max_dv, max_dur, min_stay, launch_range, max_h, max_occ)`
  - Method: `get_asteroid_trajectories(designation)` -> detailed trajectory data
  - Parse `min_dv`, `min_dur`, `n_via_traj`, observation windows
- [T1.3] Create `services/cad_client.py` - wrapper for Close Approach API
  - Method: `get_close_approaches(dist_max, date_min, date_max, body, sort, limit)`
  - Convert distances from AU to km and lunar distances (LD)
- [T1.4] Create `utils/constants.py` - astronomical constants
  - AU_TO_KM, EARTH_MU, SUN_MU, G_CONST, etc.
- [T1.5] Create `utils/unit_converter.py`
  - `au_to_km()`, `km_to_au()`, `deg_to_rad()`, `jd_to_datetime()`,

`au_to_ld()`

Week 2 Tasks

- [T1.6] Create `models/asteroid.py`, `models/orbital_elements.py`, `models/trajectory.py` SQLAlchemy models matching the schema in Part 1
- [T1.7] Create `services/data_sync.py`
  - Function: `sync_nhats_asteroids()` - fetches all NHATS targets, upserts into DB
  - Function: `sync_asteroid_details(designation)` - fetches full SBDB data, stores orbit + physical params
  - Function: `full_sync()` - runs both, logs progress
- [T1.8] Create `routers/asteroids.py`
  - `GET /api/asteroids` - paginated list from DB with sorting and filters
  - `GET /api/asteroids/{designation}` - full detail including orbit, physical params
  - `GET /api/asteroids/search?q=` - search
  - `POST /api/asteroids/sync` - trigger data_sync

Week 3 Tasks

- [T1.9] Create `utils/orbital_math.py`
  - `kepler_to_cartesian(a, e, i, om, w, ma, epoch)` -> (x, y, z) position
  - `orbital_period(a)` using Kepler's third law
  - `true_anomaly_from_mean(M, e)` - Newton-Raphson solver for Kepler's equation
  - `orbit_points(elements, n_points=360)` -> array of (x,y,z) for rendering full orbit
- [T1.10] Create `services/delta_v_calc.py`
  - Implement Hohmann transfer approximation for asteroids not in NHATS
  - `estimate_delta_v(asteroid_orbit, departure_date)` -> km/s
  - Uses vis-viva equation: v = sqrt(mu * (2/r - 1/a))
- [T1.11] Create `routers/trajectories.py`
  - `GET /api/trajectories/{designation}` - from DB or compute
  - `GET /api/trajectories/accessible?max_dv=6&max_duration=360`
  - `GET /api/trajectories/{designation}/windows`

Week 4 Tasks

- [T1.12] Create `services/mission_planner.py`
  - `generate_mission_plan(designation, launch_year, spacecraft_mass_kg)`
  - Returns: launch date, arrival date, stay duration, return date, total delta-v, fuel mass, C3, timeline phases
- [T1.13] Create `routers/mission.py`
  - `POST /api/mission/plan` - generate and save
  - `GET /api/mission/{id}` - retrieve

Week 5-6 Tasks

- [T1.14] Integrate with Member 2's EVS engine - provide trajectory data for scoring
- [T1.15] Provide orbit_points API for Member 3's 3D visualization
  - `GET /api/asteroids/{designation}/orbit-points?n=360` -> array of [x,y,z]
- [T1.16] Write unit tests for all services and orbital math
- [T1.17] Add Celery task for scheduled data sync (every 24h)

Week 7-8 Tasks

- [T1.18] Performance optimization - batch SBDB queries instead of one-by-one
- [T1.19] Error handling, logging, documentation
- [T1.20] Help Member 4 with final integration testing

MEMBER 2: ML & Economics Engineer

Owns: `backend/app/services/composition_estimator.py`, `evs_engine.py`, `market_service.py`, `routers/compositions.py`, `routers/evs.py`, `routers/market.py`, `models/composition.py`, `models/evs_score.py`, `models/commodity_price.py`

Week 1 Tasks

- [T2.1] Create `models/composition.py`, `models/evs_score.py`, `models/commodity_price.py` - SQLAlchemy models
- [T2.2] Create `services/composition_estimator.py`
  - Build the spectral-type-to-mineral mapping table (see Section 8 in Part 1)
  - Function: `estimate_composition(spectral_type, albedo, diameter_km, density)` -> dict of mineral percentages
  - Function: `estimate_mass(H, albedo, spectral_type)` -> mass in kg

    - Uses: diameter from H magnitude formula, volume from diameter, mass from volume*density
- [T2.3] Research and document commodity price sources
    - Option A: metals-api.com (free tier: 50 req/month)
    - Option B: Hardcoded recent values with manual updates
    - Platinum: ~$31,000/kg, Gold: ~$75,000/kg, Iron: ~$0.10/kg, Nickel: ~$16/kg, Cobalt: ~$29/kg, Water (in-space): ~$10,000/kg

Week 2 Tasks

- [T2.4] Create `services/market_service.py`
    - `get_current_prices()` -> dict of metal: price_per_kg
    - `update_prices()` -> fetch from API or use defaults
    - `get_price_history(metal, days)` -> for charts
- [T2.5] Create `services/evs_engine.py` V1 (hardcoded prices)
    - `calculate_evs(asteroid_id)` -> EVS score object
    - Implements the formula from Section 7 in Part 1
    - Sub-scores: accessibility (from delta-v), resource_value (from composition * prices), feasibility (from duration, OCC, spin, size)
- [T2.6] Create `routers/compositions.py`
    - `GET /api/compositions/{designation}` -> mineral breakdown + total mass + confidence
    - `GET /api/compositions/{designation}/value` -> estimated USD value per mineral

Week 3 Tasks

- [T2.7] Create `routers/evs.py`
    - `GET /api/evs/leaderboard?limit=20` -> top asteroids ranked by EVS
    - `GET /api/evs/{designation}` -> full EVS breakdown (score + sub-scores + value + cost + ROI)
    - `POST /api/evs/recalculate` -> batch recalc all scores
    - `GET /api/evs/stats` -> min, max, avg, median, distribution histogram data
- [T2.8] Create `routers/market.py`
    - `GET /api/market/prices` -> all current commodity prices
    - `GET /api/market/impact/{designation}` -> model how mining this asteroid affects supply/price
- [T2.9] Implement market impact simulation
    - For each mineral: current_global_supply, asteroid_yield, new_supply = current +

yield

- Price impact estimated via simple elasticity model

Week 4 Tasks

- [T2.10] Enhance composition estimator with ML
  - Train a simple RandomForest on known asteroid compositions (from meteorite analogs)
  - Features: spectral_type (one-hot), albedo, H magnitude, density
  - Target: mineral percentages
  - If insufficient training data, fall back to lookup table
- [T2.11] Create confidence scoring for composition estimates
  - Higher confidence when: spectral type is known (not X), diameter is measured (not estimated), density is measured
  - Lower confidence when: using H-magnitude derived diameter, no spectral type

Week 5-6 Tasks

- [T2.12] Integrate with Member 1's trajectory data for complete EVS scoring
  - Wire up delta-v and mission duration from trajectories table
- [T2.13] Create batch EVS computation for all 119 NHATS asteroids
- [T2.14] Add mission cost estimation module
  - `estimate_mission_cost(delta_v, spacecraft_mass, duration)` -> USD
- [T2.15] Write unit tests for EVS engine, composition estimator
- [T2.16] Provide EVS data format docs to Member 4 for dashboard integration

Week 7-8 Tasks

- [T2.17] Validate EVS scores against known asteroid economics literature
- [T2.18] Write user-facing documentation explaining the EVS methodology
- [T2.19] Create a "What-If" endpoint: user changes commodity prices -> recalculates EVS
- [T2.20] Help with final testing and documentation

MEMBER 3: 3D Visualization Engineer

Owns: `frontend/src/components/visualization/*`, `frontend/src/utils/orbitMath.js`, `frontend/public/textures/*`

Week 1 Tasks

- [T3.1] Setup Three.js with React Three Fiber in the Vite project

    - Install: `@react-three/fiber`, `@react-three/drei`, `three`
    - Create base `SolarSystem3D.jsx` component with Canvas
- [T3.2] Create `SunLight.jsx`
    - Central point light (warm yellow), ambient light for fill
    - Sun mesh: sphere with emissive material, slight bloom effect
- [T3.3] Create Earth, Mars meshes with textures
    - Use free NASA textures for planets
    - Earth at 1 AU on correct orbital path
    - Mars at 1.52 AU
- [T3.4] Implement `CameraControls.jsx`
    - OrbitControls from drei for mouse rotation/zoom
    - Smooth transitions when clicking on an asteroid
    - Min/max zoom limits, damping

Week 2 Tasks

- [T3.5] Create `frontend/src/utils/orbitMath.js`
    - `keplerToCartesian(a, e, i, om, w, trueAnomaly)` -> {x, y, z}
    - `solveKepler(M, e, tolerance)` -> eccentric anomaly (Newton-Raphson)
    - `generateOrbitPath(elements, numPoints)` -> array of Vector3
    - All angles in radians, distances in AU, coordinates in ecliptic frame
- [T3.6] Create `OrbitRenderer.jsx`
    - Takes orbital elements as props
    - Draws elliptical orbit as a Line geometry using orbitMath.js
    - Color-coded by EVS score (red=low, green=high)
    - Dashed line style, slight transparency
- [T3.7] Create `AsteroidMarker.jsx`
    - Small sphere at asteroid's current position (computed from mean anomaly + epoch)
    - Size scaled by estimated diameter (clamped for visibility)
    - Hover tooltip showing name + EVS score
    - Click handler to select and zoom in

Week 3 Tasks

- [T3.8] Create `TrajectoryArc.jsx`
    - Visualize Earth-to-asteroid transfer trajectory
    - Curved line from Earth's position at launch to asteroid's position at arrival

        - Animated particle traveling along the arc (spacecraft representation)
        - Color: gradient from blue (departure) to orange (arrival)
- [T3.9] Add bloom post-processing effect for the Sun
    - Use `@react-three/postprocessing` EffectComposer
    - Selective bloom on emissive materials only
- [T3.10] Implement asteroid filtering in the 3D view
    - Only show top N asteroids by EVS score (to avoid clutter)
    - Toggle to show all
    - Filter by spectral type (color-coded)

Week 4 Tasks

- [T3.11] Add info overlay panel in 3D view
    - When an asteroid is selected, show a floating glass panel with:
        - Name, spectral type, diameter
        - EVS score gauge
        - Quick stats (delta-v, estimated value)
    - Panel follows camera but stays readable
- [T3.12] Animate planetary and asteroid orbits (time controls)
    - Play/pause button to animate orbital motion
    - Time slider: current date +/- 5 years
    - Speed control: 1x, 10x, 100x, 1000x
- [T3.13] Add star background (skybox or particle field)

Week 5-6 Tasks

- [T3.14] Integrate with live backend data
    - Fetch orbit elements from `/api/asteroids/{des}/orbit`
    - Fetch orbit path points from `/api/asteroids/{des}/orbit-points`
    - Fetch trajectory data from `/api/trajectories/{des}`
- [T3.15] Performance optimization
    - Use InstancedMesh for rendering 100+ asteroid markers
    - LOD (Level of Detail) - simple dots when zoomed out, detailed spheres when close
    - Frustum culling for off-screen objects
- [T3.16] Add scale reference indicators (AU distance rings, axis labels)
- [T3.17] Mobile touch controls and responsive canvas sizing

Week 7-8 Tasks

- [T3.18] Final visual polish - glow effects, smooth transitions, loading states
- [T3.19] Screenshot/export feature for the 3D view
- [T3.20] Performance testing on low-end devices, optimize as needed

---

MEMBER 4: Full-Stack Architect (UI/UX, DB & Integration)

Owns: `docker-compose.yml`, `frontend/src/App.jsx`, `frontend/src/index.css`, all `layout/*` components, all `dashboard/*` components, all `asteroid/*` page components, all `ui/*` shared components, `frontend/src/services/api.js`, `frontend/src/store/*`, `backend/app/main.py`, `backend/app/config.py`, `backend/app/database.py`, `backend/Dockerfile`, `frontend/Dockerfile`, DB migrations

Week 1 Tasks

- [T4.1] Initialize the full repo
  - Create GitHub repo, `.gitignore`, `README.md`, `.env.example`
  - Setup branch strategy: `main`, `develop`, feature branches per member
- [T4.2] Setup backend boilerplate
  - `backend/app/main.py` - FastAPI app with CORS, lifespan, router registration
  - `backend/app/config.py` - Pydantic Settings with DATABASE_URL, REDIS_URL, env vars
  - `backend/app/database.py` - SQLAlchemy async engine, session factory, Base
  - `backend/requirements.txt` - all dependencies
  - `backend/Dockerfile`
- [T4.3] Setup frontend boilerplate
  - `npx create-vite@latest ./ --template react`
  - Install: tailwindcss, axios, react-router-dom, zustand, recharts
  - Configure `tailwind.config.js` with dark mode, custom color palette
  - `frontend/Dockerfile`
- [T4.4] Create `docker-compose.yml`
  - Services: `backend`, `frontend`, `postgres`, `redis`
  - Volumes for DB persistence
  - Environment variables from `.env`

Week 2 Tasks

- [T4.5] Create the global design system in `index.css`
    - Dark theme as default (deep space blacks: #0a0a1a, #12122a)
    - Accent colors: electric blue (#3b82f6), nebula purple (#8b5cf6), solar gold (#f59e0b)
    - Glassmorphism utilities: `.glass-card { backdrop-filter: blur(12px); background: rgba(255,255,255,0.05); border: 1px solid rgba(255,255,255,0.1); }`
    - Typography: Inter font from Google Fonts
    - Animation keyframes: fadeIn, slideUp, pulse, shimmer (loading)
- [T4.6] Create `components/ui/GlassCard.jsx` - reusable glass panel
- [T4.7] Create `components/ui/ScoreGauge.jsx` - circular gauge for EVS (0-100)
    - SVG arc that animates on load
    - Color gradient: red (0-30) -> yellow (30-60) -> green (60-100)
- [T4.8] Create `components/ui/LoadingSpinner.jsx` - orbital loading animation
- [T4.9] Create `components/layout/Navbar.jsx`
    - Logo, nav links (Dashboard, Explorer, Market, 3D Map)
    - Glass blur background, sticky top
- [T4.10] Create `components/layout/Sidebar.jsx`
    - Collapsible, contains filter controls for the dashboard

Week 3 Tasks

- [T4.11] Create `services/api.js` - Axios instance
    - Base URL from env, error interceptor, loading state management
- [T4.12] Create `store/useAppStore.js` - Zustand store
    - State: selectedAsteroid, asteroidList, evsLeaderboard, marketPrices, loading, filters
    - Actions: fetchAsteroids, fetchEVS, selectAsteroid, setFilters
- [T4.13] Create `components/dashboard/DashboardPage.jsx`
    - Grid layout: StatsCards row, TopTargetsTable, EVSDistributionChart, 3D mini-preview
- [T4.14] Create `components/dashboard/StatsCards.jsx`
    - 4 cards: Total Tracked Asteroids, NHATS Accessible, Highest EVS Score, Most Valuable (USD)
    - Each card: glass style, icon, animated counter
- [T4.15] Create `components/dashboard/TopTargetsTable.jsx`
    - Sortable table of top 20 asteroids by EVS
    - Columns: Rank, Name, Spectral Type, Diameter, Delta-V, EVS Score, Est. Value,

Action (View)

- Row click navigates to detail page
- Alternating row backgrounds, hover highlight

Week 4 Tasks

- [T4.16] Create `components/dashboard/EVSDistributionChart.jsx`
  - Histogram/bar chart of EVS score distribution using Recharts
  - Tooltip showing count per bucket
- [T4.17] Create `components/asteroid/AsteroidDetailPage.jsx`
  - Full page for a single asteroid, route: `/asteroid/:designation`
  - Sections: Header (name, class, EVS badge), Physical Params, Composition, Trajectory, Mission Planner, 3D orbit view
- [T4.18] Create `components/asteroid/PhysicalParamsCard.jsx`
  - Grid of parameter cards: diameter, density, albedo, rotation period, spectral type
- [T4.19] Create `components/asteroid/CompositionBreakdown.jsx`
  - Donut chart showing mineral percentages
  - Table with mineral name, percentage, estimated yield (kg), value (USD)
- [T4.20] Create `components/asteroid/EVSScoreGauge.jsx`
  - Large gauge showing overall EVS, sub-score bars for accessibility, value, feasibility

Week 5-6 Tasks

- [T4.21] Create `components/asteroid/TrajectoryPanel.jsx`
  - Shows: min delta-v, duration, launch window, C3, viable trajectories count
  - Integrates with Member 1's trajectory endpoints
- [T4.22] Create `components/asteroid/MissionTimeline.jsx`
  - Visual timeline: Launch -> Transfer -> Arrival -> Mining Ops -> Return -> Earth
  - Shows dates, durations for each phase
- [T4.23] Create `components/market/MarketPage.jsx`
  - Commodity price cards with sparkline charts
  - Market impact simulator (select an asteroid -> see price impact)
- [T4.24] Setup Alembic migrations and run full DB init
- [T4.25] Wire ALL frontend pages to backend APIs
- [T4.26] Implement Redis caching
  - Cache EVS leaderboard (TTL: 1 hour)
  - Cache individual asteroid data (TTL: 24 hours)
  - Cache commodity prices (TTL: 6 hours)

Week 7-8 Tasks

- [T4.27] End-to-end integration testing
    - Verify data flows: NASA API -> Backend -> DB -> API -> Frontend -> 3D
- [T4.28] Responsive design pass (tablet + mobile)
- [T4.29] Deploy
    - Backend to Render (Dockerfile, env vars, PostgreSQL addon, Redis addon)
    - Frontend to Vercel (auto-deploy from GitHub)
    - Configure CORS for production domains
- [T4.30] Final README with setup instructions, screenshots, architecture diagram

---

11. INTEGRATION DEPENDENCIES

```
M1 provides data TO:
  -> M2 (orbital elements, delta-v for EVS scoring)
  -> M3 (orbit points array for 3D rendering)
  -> M4 (asteroid list, detail endpoints for dashboard)

M2 provides data TO:
  -> M3 (EVS scores for color-coding asteroids in 3D)
  -> M4 (EVS leaderboard, composition data for dashboard/detail pages)

M3 provides components TO:
  -> M4 (SolarSystem3D component embedded in dashboard and detail pages)

M4 provides infrastructure TO:
  -> M1 (database, FastAPI app, Docker environment)
  -> M2 (database, FastAPI app, Docker environment)
  -> M3 (React app shell, routing, shared UI components)
```

Critical Integration Milestones

- **End of Week 2:** M4 delivers working Docker Compose; M1 delivers working JPL API wrappers; all members can run the full stack locally.
- **End of Week 4:** M1's asteroid endpoints return real data; M2's EVS endpoint works with hardcoded prices; M3 renders orbits from static data; M4's dashboard shows a real table.
- **End of Week 6:** All systems integrated end-to-end with live data flowing through.
- **End of Week 8:** Deployed, polished, documented.

---

12. WEEK-BY-WEEK TIMELINE SUMMARY

Week	**M1 (Astrodynamics)**	**M2 (Economics)**	**M3 (3D Viz)**	**M4 (Architect)**
1	JPL API wrappers	DB models, composition	Three.js setup, Sun/Earth	Repo, Docker, boilerplate

Week	M1 (Astrodynamics)	M2 (Economics)	M3 (3D Viz)	M4 (Architect)
		mapping		
2	DB models, data sync, asteroid routes	Market service, EVS V1, composition routes	orbitMath.js, OrbitRenderer, AsteroidMarker	Design system, UI components, Navbar
3	Orbital math, delta-v calc, trajectory routes	EVS routes, market routes, impact sim	TrajectoryArc, bloom, filtering	Dashboard page, stats, table, store
4	Mission planner	ML composition, confidence	Orbit animation, time controls	Detail page, composition chart, EVS gauge
5	Orbit points API, integration with M2	Integration with M1, batch EVS	Connect to backend APIs	Trajectory panel, mission timeline
6	Celery scheduled sync, tests	Mission cost, tests, docs	Performance optimization	Market page, Redis caching, wiring
7	Batch optimization, error handling	Validation, what-if endpoint	Visual polish, mobile	E2E testing, responsive design
8	Documentation, final testing	User-facing docs, final testing	Screenshots, final perf	Deploy, README, final integration

---

13. DEPLOYMENT CHECKLIST

- ☐ PostgreSQL database provisioned (Render/Supabase/Railway)
- ☐ Redis instance provisioned
- ☐ Backend deployed with correct env vars (DATABASE_URL, REDIS_URL)
- ☐ Initial data sync executed (populate DB from NASA APIs)
- ☐ Frontend deployed with correct VITE_API_URL pointing to backend
- ☐ CORS configured for production frontend domain
- ☐ Celery worker running for scheduled data refresh
- ☐ SSL/HTTPS enabled on both services
- ☐ README with screenshots, setup guide, architecture diagram
- ☐ Demo video recorded

---

14. RISKS & MITIGATIONS

Risk	Impact	Mitigation
NASA API rate limiting	Data sync fails	Cache aggressively in DB; sync only once per day
Three.js performance with 100+ asteroids	Laggy 3D view	Use InstancedMesh, LOD, limit visible count
Composition estimates are inaccurate	EVS scores misleading	Show confidence level; document methodology; allow manual override
Commodity price API has limited free tier	Stale prices	Hardcode fallback prices; update manually monthly
Team member availability	Delays	Clear task ownership; weekly syncs; any member can pick up UI tasks